

ANNAPURNA 360 — Full Corrected Backend

Python FastAPI backend • corrected validation, CORS configuration, matching logic, delivery validation, and dashboard calculations

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field
from datetime import datetime
from typing import List
import random
import math

app = FastAPI(
    title="ANNAPURNA 360 Backend",
    version="1.0.0",
    description="AI Food Rescue Decision and Prevention Engine"
)

# =====
# CORS
# =====
# GitHub Pages can call this API from a browser.
# For an MVP, all origins are allowed without credentials.
# In production, replace "*" with your actual frontend URL.
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"]
)

# =====
# IN-MEMORY DATABASE
# =====
batches = []
users = []
rescue_records = []

partners = [
    {
        "id": 1,
        "name": "Helping Hands NGO",
        "type": "NGO",
        "capacity": 150,
        "need": 120,
        "distance": 3.2,
        "open": True
    },
    {
        "id": 2,
        "name": "Hope Shelter",
        "type": "Shelter",
        "capacity": 100,
        "need": 80,
        "distance": 5.1,
        "open": True
    },
    {
        "id": 3,
        "name": "Community Kitchen",
        "type": "Community Kitchen",
        "capacity": 200,
        "need": 150,
        "distance": 7.4,
        "open": True
    },
    {
        "id": 4,
        "name": "Food Bank",
        "type": "Food Bank",
        "capacity": 250,
        "need": 200,
        "distance": 9.2,
        "open": True
    }
]
```

]

```
# =====
# MODELS
# =====
class DemandRequest(BaseModel):
    expected_people: int = Field(..., ge=0)
    previous_meals: int = Field(..., ge=0)
    food_type: str = Field(..., min_length=1)
    day: str = "normal"
    event_type: str = "normal"

class SurplusRequest(BaseModel):
    food_type: str = Field(..., min_length=1)
    prepared_quantity: float = Field(..., ge=0)
    consumed_quantity: float = Field(..., ge=0)
    preparation_time: str = Field(..., min_length=1)
    storage_condition: str = Field(..., min_length=1)
    source: str = Field(..., min_length=1)
    location: str = Field(..., min_length=1)

class MatchRequest(BaseModel):
    food_type: str = Field(..., min_length=1)
    quantity: int = Field(..., ge=0)
    location: str = Field(..., min_length=1)
    urgency: int = Field(50, ge=0, le=100)

class FarmRequest(BaseModel):
    produce: str = Field(..., min_length=1)
    harvest_quantity: float = Field(..., ge=0)
    market_demand: float = Field(..., ge=0)
    location: str = Field(..., min_length=1)
    harvest_date: str = ""

class WasteRequest(BaseModel):
    overproduction: float = Field(..., ge=0)
    plate_waste: float = Field(..., ge=0)
    storage_loss: float = Field(..., ge=0)
    procurement_mismatch: float = Field(..., ge=0)
    transport_loss: float = Field(0, ge=0)
    other: float = Field(0, ge=0)

class DeliveryRequest(BaseModel):
    batch_id: str = Field(..., min_length=1)
    recipient: str = Field(..., min_length=1)
    quantity: float = Field(..., gt=0)
    driver: str = Field("Volunteer", min_length=1)
    status: str = Field("Delivered", min_length=1)

class UserRequest(BaseModel):
    name: str = Field(..., min_length=1)
    role: str = Field(..., min_length=1)
    location: str = Field(..., min_length=1)

# =====
# HOME / HEALTH
# =====
@app.get("/")
def home():
    return {
        "project": "ANNAPURNA 360",
        "tagline": "Prevent • Rescue • Recover • Learn",
        "status": "Backend Running",
        "version": "1.0.0"
    }
```

```

    }

@app.get("/api/health")
def health():
    return {
        "status": "healthy",
        "server": "ANNAPURNA 360"
    }

# =====
# AI DEMAND PREDICTION
# =====
@app.post("/api/predict")
def predict_demand(data: DemandRequest):
    expected = data.expected_people
    previous = data.previous_meals

    prediction = expected * 0.70 + previous * 0.30

    if data.day.strip().lower() in {"sunday", "festival"}:
        prediction *= 1.05

    if data.event_type.strip().lower() in {
        "marriage", "function", "festival"
    }:
        prediction *= 1.08

    prediction = max(0, round(prediction))
    recommended = max(0, math.ceil(prediction * 1.05))

    saving = max(0, previous - recommended)

    return {
        "success": True,
        "food_type": data.food_type,
        "expected_people": expected,
        "previous_meals": previous,
        "predicted_demand": prediction,
        "recommended_preparation": recommended,
        "estimated_saving": saving,
        "buffer_percent": 5,
        "message": (
            f"Prepare approximately {recommended} "
            f"servings of {data.food_type}."
        )
    }

# =====
# SURPLUS DETECTION + FOOD PASSPORT
# =====
@app.post("/api/surplus")
def detect_surplus(data: SurplusRequest):
    # Prevent consumed quantity from creating a negative surplus.
    surplus = max(
        0,
        data.prepared_quantity - data.consumed_quantity
    )

    batch_id = (
        "ANN-"
        + datetime.now().strftime("%Y%m%d%H%M%S")
        + "-"
        + str(random.randint(100, 999))
    )

    storage = data.storage_condition.strip().lower()

    if storage in {"refrigerated", "cold"}:
        storage_score = 10

```

```

elif storage in {"room temperature", "normal"}:
    storage_score = 25
else:
    storage_score = 35

if surplus >= 200:
    quantity_score = 30
elif surplus >= 100:
    quantity_score = 25
elif surplus >= 50:
    quantity_score = 15
else:
    quantity_score = 5

priority = min(100, 40 + storage_score + quantity_score)

if surplus == 0:
    status = "NO SURPLUS"
elif priority >= 80:
    status = "URGENT REVIEW"
elif priority >= 60:
    status = "HIGH PRIORITY"
else:
    status = "NORMAL REVIEW"

passport = {
    "batch_id": batch_id,
    "food_type": data.food_type,
    "prepared_quantity": data.prepared_quantity,
    "consumed_quantity": data.consumed_quantity,
    "surplus_quantity": surplus,
    "preparation_time": data.preparation_time,
    "storage_condition": data.storage_condition,
    "source": data.source,
    "location": data.location,
    "priority_score": priority,
    "status": status,
    "created_at": datetime.now().isoformat()
}

batches.append(passport)

return {
    "success": True,
    "food_passport": passport,
    "safety_status": "Human review required before redistribution.",
    "flow": [
        "SOURCE",
        "SURPLUS DETECTED",
        "SAFETY REVIEW",
        "RECIPIENT MATCH",
        "DELIVERY",
        "IMPACT"
    ]
}

@app.get("/api/batches")
def get_batches():
    return {
        "success": True,
        "total": len(batches),
        "batches": batches
    }

@app.get("/api/batches/{batch_id}")
def get_batch(batch_id: str):
    for batch in batches:
        if batch["batch_id"] == batch_id:
            return {
                "success": True,

```

```

        "batch": batch
    }

    return {
        "success": False,
        "message": "Batch not found"
    }

# =====
# RESCUE PRIORITY QUEUE
# =====
@app.get("/api/rescue-queue")
def rescue_queue():
    sorted_batches = sorted(
        batches,
        key=lambda x: x["priority_score"],
        reverse=True
    )

    return {
        "success": True,
        "queue": sorted_batches
    }

# =====
# SMART RECIPIENT MATCHING
# =====
@app.post("/api/match")
def smart_matching(data: MatchRequest):
    remaining = data.quantity
    results = []

    available_partners = [
        p for p in partners
        if p["open"] and p["capacity"] > 0 and p["need"] > 0
    ]

    # Rank using need, distance and urgency.
    ranked = sorted(
        available_partners,
        key=lambda p: (
            -(
                p["need"] * 0.5
                + max(0, 100 - p["distance"] * 5) * 0.3
                + data.urgency * 0.2
            ),
            p["distance"]
        )
    )

    for partner in ranked:
        if remaining <= 0:
            break

        allocation = min(
            remaining,
            partner["capacity"],
            partner["need"]
        )

        if allocation > 0:
            match_score = (
                partner["need"] * 0.5
                + max(0, 100 - partner["distance"] * 5) * 0.3
                + data.urgency * 0.2
            )

            results.append({
                "recipient_id": partner["id"],
                "recipient": partner["name"],

```

```

        "type": partner["type"],
        "allocated_quantity": allocation,
        "distance_km": partner["distance"],
        "match_score": round(min(100, match_score), 2)
    })

    remaining -= allocation

allocated = data.quantity - remaining

return {
    "success": True,
    "food_type": data.food_type,
    "request_location": data.location,
    "requested_quantity": data.quantity,
    "allocated_quantity": allocated,
    "remaining_quantity": remaining,
    "matches": results,
    "route_status": (
        "ROUTE READY"
        if results
        else "NO RECIPIENT FOUND"
    )
}

@app.get("/api/partners")
def get_partners():
    return {
        "success": True,
        "partners": partners
    }

# =====
# FARMER / FPO ANALYSIS
# =====
@app.post("/api/farm")
def farm_analysis(data: FarmRequest):
    surplus = max(
        0,
        data.harvest_quantity - data.market_demand
    )

    if surplus == 0:
        pathway = "NORMAL MARKET"
    elif surplus <= 50:
        pathway = "LOCAL MARKET / FPO / LOCAL PROCESSING"
    elif surplus <= 200:
        pathway = "FPO / FOOD PROCESSING / INSTITUTIONAL CHANNEL"
    else:
        pathway = "FPO / PROCESSING / AUTHORIZED RECOVERY"

    utilization = []

    if surplus > 0:
        utilization.append("FPO")

    if surplus >= 50:
        utilization.append("Food Processing")

    if surplus >= 100:
        utilization.append("Other Authorized Utilization")

    return {
        "success": True,
        "produce": data.produce,
        "harvest_quantity": data.harvest_quantity,
        "market_demand": data.market_demand,
        "potential_surplus": surplus,
        "location": data.location,
        "harvest_date": data.harvest_date,
    }

```

```

        "recommended_pathway": pathway,
        "possible_channels": utilization
    }

# =====
# WASTE FINGERPRINT
# =====
@app.post("/api/waste-fingerprint")
def waste_fingerprint(data: WasteRequest):
    total = (
        data.overproduction
        + data.plate_waste
        + data.storage_loss
        + data.procurement_mismatch
        + data.transport_loss
        + data.other
    )

    if total <= 0:
        return {
            "success": False,
            "message": "Enter at least one positive waste value."
        }

    fingerprint = {
        "overproduction": round(
            data.overproduction / total * 100, 2
        ),
        "plate_waste": round(
            data.plate_waste / total * 100, 2
        ),
        "storage_loss": round(
            data.storage_loss / total * 100, 2
        ),
        "procurement_mismatch": round(
            data.procurement_mismatch / total * 100, 2
        ),
        "transport_loss": round(
            data.transport_loss / total * 100, 2
        ),
        "other": round(
            data.other / total * 100, 2
        )
    }

    main_cause = max(
        fingerprint,
        key=fingerprint.get
    )

    recommendations = {
        "overproduction":
            "Improve demand prediction and reduce over-preparation.",
        "plate_waste":
            "Improve portion control and serving management.",
        "storage_loss":
            "Improve storage monitoring and FIFO practices.",
        "procurement_mismatch":
            "Synchronize procurement with predicted demand.",
        "transport_loss":
            "Improve packaging, route planning and handling.",
        "other":
            "Collect more detailed waste information."
    }

    return {
        "success": True,
        "total_waste_units": round(total, 2),
        "fingerprint": fingerprint,
        "main_cause": main_cause,
        "recommendation": recommendations[main_cause]
    }

```

```

}

# =====
# DELIVERY
# =====
@app.post("/api/delivery")
def delivery(data: DeliveryRequest):
    # Verify the batch exists before creating a delivery record.
    batch = next(
        (b for b in batches if b["batch_id"] == data.batch_id),
        None
    )

    if batch is None:
        return {
            "success": False,
            "message": "Batch not found. Create a food passport first."
        }

    if data.quantity > batch["surplus_quantity"]:
        return {
            "success": False,
            "message": "Delivery quantity exceeds available surplus."
        }

    record = {
        "id": len(rescue_records) + 1,
        "batch_id": data.batch_id,
        "recipient": data.recipient,
        "quantity": data.quantity,
        "driver": data.driver,
        "status": data.status,
        "timestamp": datetime.now().isoformat()
    }

    rescue_records.append(record)

    batch["recipient"] = data.recipient
    batch["transfer_status"] = data.status
    batch["delivered_quantity"] = (
        batch.get("delivered_quantity", 0) + data.quantity
    )
    batch["remaining_quantity"] = max(
        0,
        batch["surplus_quantity"] - batch["delivered_quantity"]
    )

    return {
        "success": True,
        "message": "Delivery record created",
        "delivery": record
    }

@app.get("/api/deliveries")
def get_deliveries():
    return {
        "success": True,
        "total": len(rescue_records),
        "deliveries": rescue_records
    }

```

```

# =====
# USER REGISTRATION
# =====
@app.post("/api/users")
def create_user(data: UserRequest):
    user = {
        "id": len(users) + 1,
        "name": data.name,

```

```

        "role": data.role,
        "location": data.location,
        "created_at": datetime.now().isoformat()
    }

    users.append(user)

    return {
        "success": True,
        "user": user
    }

@app.get("/api/users")
def get_users():
    return {
        "success": True,
        "total": len(users),
        "users": users
    }

# =====
# IMPACT DASHBOARD
# =====
@app.get("/api/dashboard")
def dashboard():
    rescued = sum(
        b["delivered_quantity"]
        for b in batches
    )

    deliveries = len(rescue_records)

    return {
        "success": True,
        "metrics": {
            "meals_rescued": 1240 + int(rescued),
            "waste_prevented_kg": 380 + int(rescued * 0.25),
            "rescue_actions": 72 + deliveries,
            "material_recovered_tonnes": round(
                2.8 + rescued / 1000,
                2
            )
        },
        "district": {
            "surplus_identified_tonnes": round(
                18.2 + sum(
                    b["surplus_quantity"] for b in batches
                ) / 1000,
                2
            ),
            "food_rescued_tonnes": round(
                12.4 + rescued / 1200,
                2
            ),
            "food_prevented_tonnes": 3.8,
            "permitted_recovery_tonnes": 1.2
        },
        "waste_pressure": [
            {"area": "Area A", "level": "HIGH"},
            {"area": "Area B", "level": "MEDIUM"},
            {"area": "Area C", "level": "LOW"}
        ]
    }

# =====
# RECOVERY OPTIONS
# =====
@app.get("/api/recovery-options")
def recovery_options():

```

```

return {
    "success": True,
    "flow": [
        "Surplus Identified",
        "Safety / Rule Check",
        "Human Redistribution",
        "Permitted Recovery"
    ],
    "recovery_options": [
        {
            "name": "Human Redistribution",
            "icon": "■",
            "priority": 1
        },
        {
            "name": "Food Processing",
            "icon": "■",
            "priority": 2
        },
        {
            "name": "Animal Feed",
            "icon": "■",
            "priority": 3,
            "condition": "Only where legally permitted."
        },
        {
            "name": "Composting",
            "icon": "■",
            "priority": 4
        },
        {
            "name": "Other Authorized Recovery",
            "icon": "■■",
            "priority": 5
        }
    ]
}

# =====
# SYSTEM SUMMARY
# =====
@app.get("/api/system")
def system_summary():
    return {
        "name": "ANNAPURNA 360",
        "tagline": "Prevent • Rescue • Recover • Learn",
        "modules": [
            "AI Demand Prediction",
            "Surplus Food Detection",
            "Food Passport",
            "Rescue Priority Queue",
            "Smart Recipient Matching",
            "Route Planning",
            "Farmer / FPO Mode",
            "Waste Fingerprint",
            "Food Recovery",
            "Delivery Tracking",
            "Impact Dashboard"
        ],
        "architecture": {
            "frontend": "HTML CSS JavaScript",
            "backend": "Python FastAPI",
            "database": "In-memory MVP",
            "ai": "Explainable prediction logic",
            "maps": "Ready for routing API integration"
        }
    }

# =====
# RUN SERVER

```

```
# =====  
if __name__ == "__main__":  
    import uvicorn  
  
    uvicorn.run(  
        app,  
        host="0.0.0.0",  
        port=8000  
    )
```